

Welcome to Hawkins Experiments

Goal of this project

Welcome to the **Hawkins Experiments Intro Course**.

This course teaches you to understand how to create **Code-Based (CB) tasks** that can be **deterministically built and evaluated**. Your job in Hawkins is to provide a complete task package (prompt + Docker + scripts + golden patch) so an automated system can run validation consistently.

Rules for the Course

- Any **free-text** responses must be written in **English**.
- Read each question carefully; assume you're working on a real submission.

Project Overview and Required Deliverables

What is Hawkins Experiments?

The purpose of this project is to create a task structure that will evaluate an AI model's ability to make changes to a code repo that fix a specific problem **without introducing regressions** into the code base.

You will do this by creating a prompt that asks the model to carry out a specific action or solve a specific problem within a GitHub repo. Then, you will **create an environment** that can be used to **grade the model's response** to the prompt. You will not actually see the model's response in these tasks, but are just building the environment that can be used to grade it.

There are two different task types that you will be working with: Refactoring/ Maintainability (R/M) tasks and Performance Optimization (PO) tasks.

Depending on the task type that you select, the evaluation environment that you create will be slightly different.

Required deliverables

These are the components that you will create for each task.

All Task Types

- **User-facing problem description**
- **Dockerfile**
- **Prompt**
- **Run Script**
- **Parsing file**
- **Golden Patch**

In addition to the components shown above, you will create the components shown below for the task type that you select.

Refactoring/Maintainability (R/M) Tasks:

- **Grading Rubric**

Performance Optimization (PO) Tasks:

- **Target Functions**
- **Reproduction Script**

Common Steps 1–4 (Source → Prompt → Docker → Task Type)

Step 1: Understand the source (source_url)

You will be provided with a GitHub link that will act as the source for your task. This link will lead to a GitHub PR.

You must open the provided link and understand the issue that is being solved by the PR. This issue is what you will use to write the user-facing problem description and the prompt for the task.

Step 2: Configure Dockerfile (deterministic environment) and provide task metadata

1. Write a User-Facing Problem Description

- The user-facing problem description is a brief description of the problem that is presented in the provided GitHub link. Understand what issue the GitHub PR solves, and then create a brief summary or description of that issue as if it hasn't been solved yet.

2. You will create a Dockerfile that must:

- Clone/build the repo
- Install dependencies

2. You will create a Dockerfile that must:

- Clone/build the repo
- Install dependencies
- Checkout the parent commit of the PR
- Be reproducible

3. Identify the Primary Coding Language

- Identify the main programming language that will be used to solve the issue.

Dockerfile Guidelines

The Dockerfile that you create should follow the template provided in the task resources. **ONLY** modify the sections that contain a ``#TODO:`` comment. Make sure that the Dockerfile:

- Checks out the **parent commit** of the Golden Patch.
- Checks out the commit using a specific commit hash, not a branch name.
- Installs ALL libraries and dependencies needed to run the `reproduction_script.sh` and `run_script.sh` files.
- Does NOT specify or hard code a platform architecture (eg. `x86_64`, `AArch64`, etc.). It needs to be platform agnostic.
- Does NOT run `COPY` commands for any of the `run_script.sh`, `reproduction_script.sh`, `golden.patch`, or `parsing.py` files.

Step 3: Write the prompt (what the model sees)

Create the prompt that describes the issue that the PR solves. Do not directly reference the PR, the `run_script`, the `reproduction_script`, the Golden Patch, the rubrics, etc., only describe the issue and request that it be fixed. Prompts should not be written in second person.

Prompt Requirements:

- **Self-contained:** All necessary information is contained in the prompt
- **Clear constraints:** What must/must not be changed
- **Defined "done":** How to know when the task is complete
- **Preserves Functionality:** Do not include a request in the prompt that changes the observable functionality of the code.
- **No solution:** Must NOT include the solution or direct hints

The following prompt template should be used:

PLAINTEXT



Collapse ^

Context

[Brief background on the codebase and the problem]

Problem Statement

[Clear description of what needs to be fixed/improved]



Context

[Brief background on the codebase and the problem]

Problem Statement

[Clear description of what needs to be fixed/improved]

Requirements

1. [Specific requirement 1]
2. [Specific requirement 2]
3. [Constraint - what NOT to change]

Success Criteria

- [Measurable criterion 1]
- [Measurable criterion 2]
- [For PO tasks, include the performance that should be improved]
- [All previously passing tests must pass]

Files to Modify

- `path/to/file1.py`
- `path/to/file2.py`

Notes

[Any additional context, but NO hints toward the solution]

Step 4: Choose the task_type

The task type that is chosen will determine the type of evaluation that will be used to verify the quality of the solution to the issue.

- **Refactoring** → Grading Rubric + Unit Tests
- **Performance Optimization** → Quantitative Metrics + Unit Tests

The task will show a suggested task type:

- **The Task Type will be:** Refactoring/Migration

The suggested task type may or may not be correct. You must verify and then select the official task type for the task based on the issue in the PR and the prompt that you write.

You will then select the correct sub-type or category that best describes the issue.

- ☒ Performance Optimization (PO)
- ☐ Refactoring/Maintainability (R/M)

[PO] Optimization Category *

Select the specific optimization type (Performance Optimization tasks only)

- ☐ Algorithmic
- ☐ Memory
- ☐ I/O
- ☐ Vectorization

All task types must include:

Run Script

The run script should run a set of unit tests that should be taken from the existing test suite within the GitHub repo you are working with. The run script should ONLY run the tests that already pass before the Golden Patch is applied.

The purpose of this script is to verify that the Golden Patch does not introduce regressions into the code. You will run this script before applying the Golden Patch to make sure that all tests pass. After that, you will apply the Golden Patch and then you will rerun this script to make sure that the same tests are all still passing.

Script Requirements:

- Run build/compile process (if applicable)
- Run linting checks (if applicable)
- Run existing passed unit tests

Find template run script here: <https://app.outlier.ai/en/expert/guidelines/697aa99a5d1ac05a38db7e20?componentId=69b19c0041e6fc11d2c27721&type=richText>

Parsing.py File

This is a file that takes in the results (stdout.txt and stderr.txt) of the run script (run_script.sh), parses them, and outputs a JSON file that follows the correct format. This happens automatically when you run e2e.sh (shown below). Use the parsing.py template that is given in the project resources. It implements most of the logic, but you will need to modify the `parse_test_output` function. Only modify the `parse_test_output` function.

Template Script (parsing.py)

FIND HERE : <https://app.outlier.ai/en/expert/guidelines/697aa99a5d1ac05a38db7e20?componentId=69b19ffcfd730684dd4241d&type=richText>

Golden Patch

Implement the "Gold Standard" solution as a diff patch. The Golden Patch should contain the changes made to the code repo in order to fully answer the prompt. The Golden Patch should be the perfect solution to ALL of the requests in the prompt.

The Golden Patch should only contain the changes necessary to answer the prompt, and should not include additional changes that go beyond the scope of the prompt.

The Golden Patch should follow best coding practices, with appropriate documentation (code comments, docstrings, etc), good naming conventions, and logical structures and formatting.

NOTE: You can only use the diff patch provided in the GitHub PR if it represents the perfect solution to your prompt. Otherwise, you must create the Golden Patch yourself.

The Golden Patch should follow best coding practices, with appropriate documentation (code comments, docstrings, etc), good naming conventions, and logical structures and formatting.

NOTE: You can only use the diff patch provided in the GitHub PR if it represents the perfect solution to your prompt. Otherwise, you must create the Golden Patch yourself.

Requirements:

- Must be a valid unified diff format
- Must have a .patch extension
- Must apply cleanly to the commit that is checked out in your Dockerfile
- Must pass ALL validation (e2e.sh) scripts
- Must fully solve the problem stated in the prompt
- Should follow best coding practices
- (R/M Tasks) Must pass ALL rubric criteria

Refactoring/Maintainability Tasks

Along with the other files that all tasks need, each task type has additional requirements that must be followed.

Refactoring/Maintainability Tasks must include:

Rubrics

Write clear rubrics that can be used to grade a response to your prompt. The rubric should be written based on the prompt that you created. The rubric should be a comprehensive, self-contained evaluation of how well a response fully fulfilled all prompt requests.

There will be an LLM rubric generator to help build the rubric criteria. Those LLM-generated rubrics **MUST** be revised thoroughly and edited to fit the guidelines and requirements.

WARNING: The rubric-generating model can make mistakes that must not be overlooked. These LLM-generated rubrics should not be treated as a ground truth, but as a starting point and inspiration for the complete set that you must create. You will need to modify, add, and delete criteria from the LLM-generated rubric in order to create the perfect rubric that you will submit with the task.

DISCLAIMER: Leaving the rubrics as is, without fine-tuning them, will result in being disabled from the project.

DISCLAIMER: Leaving the rubrics as is, without fine-tuning them, will result in being disabled from the project.

Expert Rubric

A **rubric** is a set of clear, specific requirements that describe what a strong response to your prompt must contain. You must create a rubric document tailored specifically to your prompt's requirements with a **minimum of 15 criteria and a maximum 20 criteria**.

Rubric Requirements:

- Must comprehensively cover ALL explicit and implicit requests in the prompt that aren't already checked in the `run_script`, `reproduction_script`, `Dockerfile`, or `e2e.sh`. If the rubric fails to evaluate something that is requested in the prompt and is not checked by other files, your task will fail.
- Must accurately evaluate ALL possible correct solutions to the prompt, not only your specific Golden Patch.

Criteria Requirements:

- **Rubric-Specific:** Rubric criteria must only check parts of the solution that are not already verified in the `run_script`, `reproduction_script`, `Dockerfile`, or `e2e.sh`. For example, you must NOT include any rubric criteria that talk about unit tests passing, since that is already verified in the `run_script`.
- **Atomic:** Each criterion checks only one idea. A criterion should not evaluate multiple, unrelated things.
- **Objective:** Each criterion must objectively evaluate a specific part of a response. When grading a response, you should be able to "PASS" or "FAIL" each criterion objectively, without any subjective ambiguity.
- **Correctly Framed:** Each criterion should be written in such a way that a good response would evaluate to "Yes" or "True" in order for that criterion to "PASS".

- **Objective:** Each criterion must objectively evaluate a specific part of a response. When grading a response, you should be able to "PASS" or "FAIL" each criterion objectively, without any subjective ambiguity.
- **Correctly Framed:** Each criterion should be written in such a way that a good response would evaluate to "Yes" or "True" in order for that criterion to "PASS".
- **Positive:** Uses positive phrasing (e.g., "Code includes..." vs "Code doesn't forget...").
- **Accurate:** Each criterion should accurately evaluate part of the prompt. For example, if the prompt says, "Include 3 new functions that do...", then you should NOT have a criterion that says, "The solution includes 4 new functions that do...".
- **Scoped:** Each criterion should evaluate something that is explicitly or implicitly requested in the prompt. Do not include any criteria that check for something that is not requested in the prompt.
- **Self-Contained:** Each criterion needs to be able to be evaluated ONLY within the context of the criterion itself and the response diff patch. In order to accurately grade the criterion, you should not need to reference the prompt, the PR, any of the scripts, or any other outside material. Include all information that is needed to evaluate the criterion within the criterion itself. (Bad Example: "The solution correctly implements the sorting logic on the input array." vs. Good Example: "The solution implements merge sorting logic on the array that is received as input in the ``test_name`` function.")
- **Unique:** Each criterion should evaluate something unique. There should be no redundancy or overlapping between multiple criteria.
- **Not Overfitting or Underfitting:** Criteria should be able to accurately grade ALL correct solutions based on the requirements and requests in the prompt. Avoid writing criteria that are overly specific to the point that they would fail a good response that accurately follows the prompt. On the flip side, avoid writing criteria that are so generic that they pass a bad response that does not follow the prompt.

Rubric Weights

Each rubric criterion must have **one** of the following weights:

- **5 — Mandatory** This is a core requirement for adequately answering the prompt, is clearly expected based on the prompt, and it would be hard to envision an acceptable response without it.
- **3 — Important** This makes the response substantially better, and is reasonably or implicitly expected based on the prompt, though you can envision an acceptable response without it as long as all other criteria are met.
- **1 — Nice to have (Optional)** This is a nice-to-have and necessary for a perfect response, but a response can still be strong without it.

⚠ Do NOT select 2 or 4 as weights. The only allowed options are 1, 3, or 5. ⚠

Formatting:

Rubric criteria should follow these formatting guidelines:

- Should NOT be written in past tense.
- Should NOT be written as a question.
- Should NOT begin with: "The model..."

Scoring Rules

For each rubric item, you should be able to determine if a given response is a "PASS" or a "FAIL".

- **Criteria Fully Met (PASS)**
- **Not Met (FAIL)**

Rubric criteria should follow these formatting guidelines:

- Should NOT be written in past tense.
- Should NOT be written as a question.
- Should NOT begin with: "The model..."

Scoring Rules

For each rubric item, you should be able to determine if a given response is a "PASS" or a "FAIL".

- **Criteria Fully Met (PASS)**
- **Not Met (FAIL)**

NOTE: Your Golden Patch MUST have all criteria as Pass

Criteria Categories

1. **Instruction Following:** Ensures the response adheres to explicit directions in the prompt (format, constraints, language, libraries, required elements).
2. **Code Correctness:** Ensures any code performs the intended task and produces correct results based on the prompt.
3. **Code Clarity:** Covers readable, well-structured code, including naming, organization, and formatting.
4. **Efficiency:** Covers conciseness, avoidance of unnecessary steps, and reduction of redundancy.
5. **Risk Security:** Ensures a safe response that avoids inappropriate or harmful content.

Performance Optimization Tasks

Performance Optimization Tasks must include:

Reproduction Script

Write a script that measures the performance of the relevant code. The relevant code is the code that contains the performance bottleneck in the repo, and that must be optimized in the Golden Patch in order to improve performance. This script will test that the optimization provided in the Golden Patch has tangible, reproducible, and quantifiable improvements.

You will need to determine the best way to measure this performance. The performance metric could measure speed, memory, latency, operations per second, etc. The benchmark should evaluate a relevant performance metric that can be used to grade a response to your prompt. In other words, if a response to your prompt is high-quality, then the reproduction script should show a tangible, reproducible, and quantifiable improvement in the output performance metric when that response is applied to the repo.

NOTE: Your reproduction script MUST test the actual, relevant code that contains the performance bottleneck before the golden patch and is optimized after the golden patch. Creating simulated or dummy benchmarks that don't actually run the relevant code in the repo is strictly prohibited.

Script Requirements:

- Run build/compile process (if applicable)

Script Requirements:

- Run build/compile process (if applicable)
- Create performance benchmark test
- Run performance benchmark test
- Output a single, parseable metric in this format: "METRIC_VALUE: "
- Run deterministically

Target Functions

List the specific function(s) that must be optimized. These are the existing functions in the code base that the model must identify as the ones needing modification in order to improve the performance of the code.

Requirements:

- Must be specific function names with file paths
- Functions must be the actual performance bottleneck
- Include class name if applicable

Example Format:

PLAINTEXT



```
"src/data/processor.py::DataProcessor::process_batch",  
"src/utils/parser.py::parse_large_file"
```

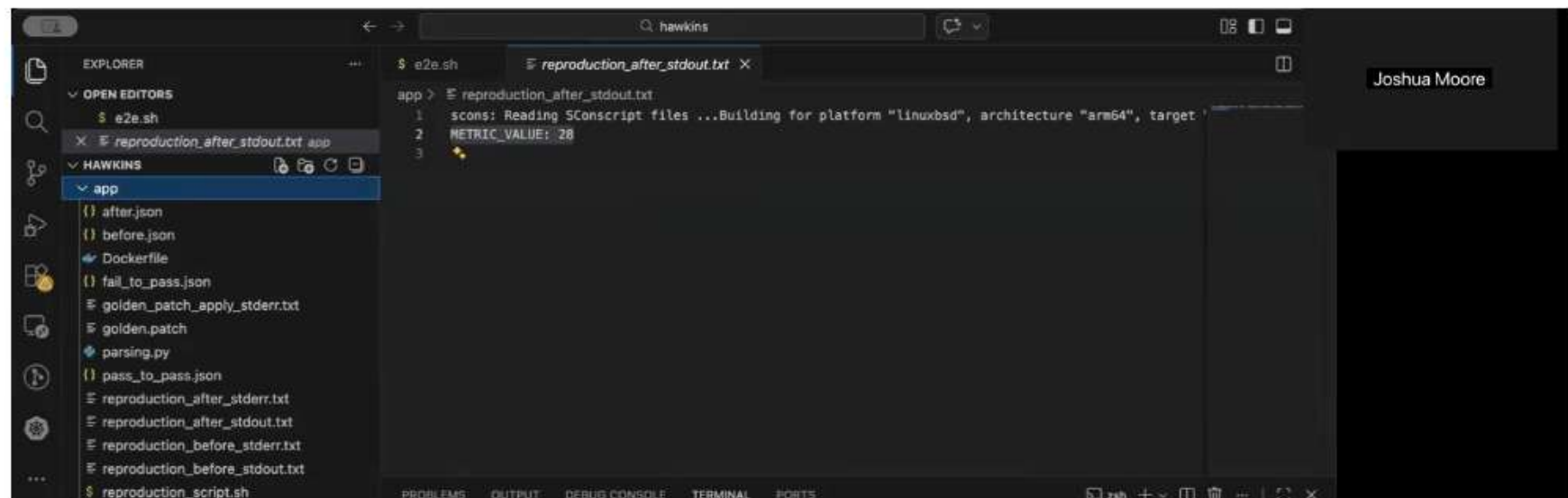
Final Steps: Self-Check, Common Mistakes

End to End Script

For every task, you will have to run an end-to-end (e2e.sh) script that validates all of the files in your task.

**You can find the e2e.sh script here: <https://app.outlier.ai/en/expert/guidelines/697aa99a5d1ac05a38db7e20?componentId=699e87e13ff45e697c58ca9d&type=richText>

Watch the following video to see how to use this script:



NOTE: You CANNOT make any changes to the e2e.sh script. It must run fully without any errors and must produce the required output in order for you to submit your task. If the script does not run fully without errors or does not produce the correct output, then you must change YOUR files until it does.

The end-to-end script does the following:

- Builds a Docker image with your Dockerfile
- Runs the Docker container
- Runs the reproduction_script.sh file to measure pre-patch performance
- Runs the run_script file to make sure tests are passing
- Runs parsing.py with the output of the run_script
- Applies the golden patch
- Runs the reproduction_script.sh file to measure post-patch performance
- Runs the run_script file to make sure the same tests are still passing
- Runs parsing.py with the output of the run_script
- Creates several new files with output (see project instructions for more details)

Final self-check (before submission)

Verify:

- Docker image builds successfully
- e2e.sh script runs without errors
- Gold patch applies cleanly
- Golden patch fully answers the prompt
- (R/M) Golden patch passes all rubric criteria
- (R/M) Rubric follows all project guidelines (see project instructions)
- All tests pass before the gold patch is applied
- All tests pass after the gold patch is applied
- before.json and after.json are identical
- (PO) Reproduction script runs before the gold patch is applied
- (PO) Reproduction script runs after the gold patch is applied
- (PO) Reproduction script shows tangible, reproducible, and quantifiable improvements.

Common mistakes to avoid

- Putting solution hints in the prompt
- Using branch names instead of commit hashes
- Non-deterministic benchmarks
- Vague success criteria
- Overly broad scope